

Sameer Khan

Senior ML Engineer | Document AI & Multimodal | LLM / RAG | MLOps on AWS

+91-

8096796963 | sameerkhanofficial9@gmail.com | linkedin.com/in/mrsameerkhan3 | github.com/MrSameerKhan | Hyderabad, India

SUMMARY

Senior Machine Learning Engineer with **9 years in Document AI** — computer vision, NLP and multimodal deep learning applied to one problem across three roles, turning scanned documents into structured, reliable data. Currently own ICE's production mortgage page classifier: **643 classes at ~1.7M requests/day** on SageMaker with **99.99% success**, and I proposed and built its **OCR-free transformer successor**. Also built near-duplicate detection across a 7.1M-page corpus, and a bank's document-digitisation platform as its first ML engineer. Hands-on across the full lifecycle — model architecture, distributed training, the serving path, and production incident response. Targeting senior individual-contributor roles with architecture ownership.

TECHNICAL SKILLS

ML / Deep Learning: PyTorch, PyTorch Lightning, TensorFlow / Keras, Hugging Face Transformers, CNN, ResNet50, Swin / Vision Transformers (ViT), Donut, multimodal fusion, BiLSTM, NER, full-parameter and parameter-efficient fine-tuning (LoRA / QLoRA / PEFT), SFT / instruction tuning, RLHF-DPO, mixed precision (fp16 / bf16)

Distributed Training & Data: Horovod, DDP / TorchDistributor, NCCL, multi-GPU training, Spark / PySpark, Databricks

Document AI / Computer Vision: document classification, information extraction, OCR and OCR-free processing, document deduplication, signature verification, object detection, image preprocessing, Tesseract, AWS Textract, TensorFlow Object Detection, OpenCV

LLM / RAG: RAG, FAISS, dense retrieval, Okapi BM25, hybrid retrieval, reciprocal rank fusion (RRF), cross-encoder reranking, structured outputs (JSON Schema), selective prediction, confidence calibration, Llama 3, Anthropic Claude API

Serving / MLOps: AWS SageMaker (real-time endpoints, autoscaling, model registry), TF Serving, ONNX, model optimisation, Jenkins CI/CD, Docker, MLflow, CloudWatch, production monitoring

Languages / Tools: Python, SQL, Git, Linux / bash

WORK EXPERIENCE

- Own ICE's production mortgage document classifier end to end. **643 classes**, trained on **2.8M pages** from 49 encrypted lender sources. It serves **1.7M requests/day (45M/month)** on SageMaker at **p95 1.8s** with **99.99% success**, and cut loan-file document verification from about 4 days down to 4–5 hours.
- Designed the **five-branch multimodal architecture**: ResNet50 plus CNN, BiLSTM, and a **multi-head-attention Transformer I wrote from scratch** over frozen GloVe. Trained it on **Horovod across 20x A10G GPUs**, after fixing the host-RAM OOM and all-reduce failures that were blocking runs. It gets **weighted F1 0.959 to 0.972** across five lender benchmarks.
- **Proposed and built the OCR-free successor** to get OCR out of the inference path: **Donut / Swin encoder, 75M parameters fully fine-tuned**, PyTorch Lightning DDP on **20x L40S** at bf16. It hit **weighted F1 0.975** on 528,574 pages against the OCR-based production ensemble's 0.974. I exported it to **ONNX**, benchmarked 9 SageMaker instance types and picked one **~20% cheaper per hour**.
- **Led the P1 root-cause analysis that cleared the model**. 21,096 pages were reported failed in a 260K-page regression. I traced 5 services in CloudWatch Logs Insights and ruled out both the model and the outage hypotheses. The real cause was in the platform Java path: an input validator was rejecting whole documents before inference, so **one dropped page could null a 530-page file**. True failure rate was **2.3%**, and the platform team shipped the fix I recommended.
- Recovered a production accuracy regression across **946,355 pages**. Fixed a train/benchmark taxonomy mismatch that **took F1 from 0.62 to 0.92**, then found 64% of what was left was correct predictions being thrown away by one confidence threshold. I own the per-class threshold config — **calibrated selective prediction** across all 643 classes.
- Shipped **four consecutive production model releases** through the Jenkins-to-SageMaker pipeline: cross-account artifact promotion, model registration, previous-version teardown, and one rollback. Handled autoscaling from **20 to 118 instances** under burst and took every release from staging to production through change control. Also migrated the serving export from **TF1 to TF2**.

Persistent Systems (Client: ICE Data Services) | Senior ML Engineer | Hyderabad

Aug 2021 – Nov 2023

- Built the data-ingestion and OCR layer that the production classifier still runs on. Decrypted lender image sets, extracted text at scale with **Tesseract**, and normalised **27+ source schemas** into one labelled corpus using PySpark UDFs.
- Added the **OpenCV preprocessing stage** — thresholding, denoising, skew correction, normalisation — that raised OCR quality for every downstream model.
- Built the first multimodal **CNN + BiLSTM** training pipeline on Databricks, including the BiLSTM encoder-decoder that produces the sequence representations the text branches consume. The production 643-class ensemble was scaled up from this.
- Set up the initial **SageMaker** deployment and monitoring path plus **MLflow** experiment tracking. The current Jenkins-to-SageMaker release pipeline was built on top of that.

- Designed and built the bank's document-digitisation pipeline **as its first ML engineer**, running over **~100,000 bilingual Arabic and English documents** end to end — classification, field detection, OCR and entity extraction.
- Built the model chain: a **ResNet50** classifier over **~65 document types (~88% accuracy)**, TensorFlow Object Detection to localise the required fields, then region cropping, Tesseract OCR, and a **GRU-based NER** to pull out entity values (name, country, ID number).
- Built a deep-learning **signature-verification model** to flag forged signatures.
- Traced the pipeline's largest error source upstream to the branches, each scanning at different settings — colour distortion, rotation and skew. I audited scanner configurations on-site across the bank's major branches and derived a standard profile, and **the bank issued it as a circular mandating those settings bank-wide**. That fixed the problem at the source instead of in preprocessing.
- Integrated extraction output into the bank's RPA (BluePrism) workflow behind a **human-in-the-loop review gate**. Every page and its extracted values were checked before commit, and corrections went back as training signal. Shipped **on-premise** as Flask + Gunicorn microservices, so no document left the bank's network.
- **Mentored 5+ engineers** as the team grew around the pipeline I had built.

VISSINDIA | Python & Computer Vision Engineer | Hyderabad

Feb 2017 – Aug 2019

- Built real-time computer-vision pipelines for identity verification and multi-object tracking: face detection (Haar cascades, HOG, CNN), keypoint matching (SIFT / SURF / FLANN), and TensorFlow object detection across video frames. Also built the OpenCV preprocessing layer that made noisy imagery usable by the detection models.

PROJECTS

Rulebook-RAG | Retrieval-grounded document classification — personal project, 2026

- Built a **36-class** document classification system with **no trained model in the loop and no labelled training data**. The class knowledge sits in a written rulebook generated from official instruction documents, and gets retrieved per query. To add a class I write one rule and rebuild the index: **0.83 accuracy** on 5 classes added after the index was built, with no retraining.
- **Hand-wrote Okapi BM25** and fused it with dense embeddings using reciprocal rank fusion (**+5.2 pts**), then added cross-encoder reranking (**+3.9 pts**). Every prediction cites a rule id, and each id is resolved against the live index before the answer is returned: **0 invalid of 775**.

Document Deduplication Pipeline — ICE Data Services

- Built near-duplicate detection across a **7.1M-page** mortgage corpus. Generated DonutSwin page embeddings with **TorchDistributor across 40 GPU nodes**, then ran a two-tier **FAISS IndexFlatIP** cosine search — one pass at page level, one on mean-pooled document embeddings.
- Ran the searches inside label and page-count groups **so I could use many small exact searches instead of one approximate index**, and used **Union-Find** to collapse transitive duplicate chains. Flagged **67% of pages / 69% of documents** as near-duplicates.

EDUCATION
